RNAP Hack 2026

Challenge Tracks · Redes Neuronales y Aprendizaje Profundo · November 2026

About the event

RNAP Hack 2026 runs for three Thursdays: November 5, 12 and 19. Teams of up to 4 students build a working language product with what the course has covered, from text representation and recurrent networks to attention, Transformers and pre-trained models.

All the work happens during class time and every checkpoint goes to GitHub. On Thursday November 19 every team presents its product in English and lets the judge use it live.

Each team picks one of the five tracks below. Several teams can work on the same track.

> *Other documents of the kit: Calendario, Guía de GitHub, Instrucciones de Redes Neuronales and English Instructions. The last two describe every deliverable and how it is evaluated.*

Common rules for every track

Every team, whatever its track, follows the same base so the judge can compare solutions fairly.

- **Two levels of the course.** Every solution compares a baseline built from scratch with early course tools (TF-IDF with a classic classifier, a small LSTM or rules) against a solution built on a pre-trained Transformer.
- **Attention in the demo.** The team explains one element related to attention in its own model, such as an attention map or the encoder-decoder structure.
- **Clean data split.** Split the data into train, validation and test before training. Use the test set only for the final metrics.
- **Interface in Gradio.** During class, run it from Colab with `share=True`. The final version runs on Hugging Face Spaces (free CPU) or on the team’s laptop, and the judge must be able to use it.
- **AI tools are allowed** for writing code. Every member must be able to explain the project, and the judge asks the team surprise questions.
- **Data.** Use only public, free data or data your team creates. Label synthetic data as synthetic.
- **Keep it small.** One flow that works from start to end scores higher than several half-finished features.

Tracks at a glance

Track	Task	Main metric
1. Human or Machine?	Classify a message as written by a person or by a language model	F1
2. News in 60 Seconds	Summarize a Spanish news article in 120 words or less	ROUGE and word limit
3. From Voice Note to Order	Turn a Spanish voice note into an order ticket	WER and correct fields
4. The Assistant That Doesn’t Make Things Up	Answer questions about a regulation, quoting the article	Right article in top 3 and traps refused
5. The Interface That Follows Your Intent	Detect the intent of a sentence and show the right screen card	Intent accuracy and F1

Data sources

- AuTexTification 2023: huggingface.co/datasets/symanto/autextification2023 · overview paper: huggingface.co/papers/2309.11285
- XL-Sum: huggingface.co/datasets/csebuetnlp/xlsum
- MASSIVE: huggingface.co/datasets/AmazonScience/massive
- Whisper: huggingface.co/openai/whisper-small

Track 1 — Human or Machine?

About the company. ChatSeguro is a customer-service company in Monterrey. It handles chat support for insurance clients across Mexico. (Fictional company.)

The problem today. Over the last year, bots powered by language models have started to pose as customers. They have opened fake claims and have asked agents for policy data. The agents have read thousands of these messages, and they haven’t found a reliable way to tell who is writing. If the agents had known which chats came from bots, they would have stopped most of the fake claims on the first message.

The big challenge. Given one message, can your system tell whether a person or a language model wrote it, and how sure is it?

Possible approaches.

1. **Word patterns.** TF-IDF with logistic regression, or a small LSTM, as the baseline.
2. **Fine-tuned Transformer.** Fine-tune a Spanish BERT such as BETO (`dccuchile/bert-base-spanish-wwm-cased`) or a multilingual DistilBERT on the labeled texts.
3. **Predictability.** Text from a language model is usually easy for another model to predict. Measure perplexity with a small Spanish GPT-2 (for example `DeepESP/gpt2-spanish`) and use it as an extra signal.
4. **Semantic trap.** In a chat, ask about something that doesn’t exist. A person usually says they don’t know it, and a model often invents an answer. This is an optional extra.

These are directions, not requirements. One signal done well scores higher than several that half-work.

Resources. AuTexTification 2023 (IberLEF), Spanish detection subtask, at huggingface.co/datasets/symanto/autextification2023. It has 32,062 training texts from five domains labeled as human or generated. Download the files from the Files tab of the dataset. Its generators are from 2023 (BLOOM and GPT-3 models), so add at least 50 texts from current chatbots and 50 written by your team to test robustness. License CC BY-NC-SA 4.0.

What to build. An interface with a text box. It returns the verdict (human or machine), a confidence from 0 to 100% and the words or signals that weighed most in the decision.

Judge test. The judge pastes texts your team has never seen, some written by the judge and some by current chatbots.

Why it matters. If companies couldn’t tell people from bots, every chat channel would become a door for fraud. A detector that works with current models would let a company keep chat support open and safe.

Judging criteria.

- **Results:** F1 on the judge’s unseen texts.
- **Robustness:** how well it handles text from current chatbots.
- **Explanation:** the team shows why the model decided what it decided.
- **Clarity:** a person who has never used the interface understands the verdict.

Track 2 — News in 60 Seconds

About the company. Radio Cumbres is a local radio station in Nuevo León with a morning news show. (Fictional company.)

The problem today. The host has always had 60 seconds per story. Since the show started, two producers have summarized every story by hand, starting at 5:00 a.m. They have tried online summarizers, but those tools have produced texts that are too long or that add facts. Last month a summary included a wrong figure. If the producer had compared it with the original article, the host wouldn’t have read that number on air.

The big challenge. Can your system turn a Spanish news article into a summary that a host can read aloud in 60 seconds, using only facts from the article?

Possible approaches.

1. **Extractive baseline.** Score each sentence with TF-IDF and keep the best ones until you reach the word limit.
2. **Ready summarizer.** Use `csebuetnlp/mT5_multilingual_XLSum`, an encoder-decoder model already fine-tuned for summaries in many languages, including Spanish.
3. **Language model with a prompt.** Use a free API or a local model with Ollama, with clear rules about length and facts.
4. **Fact guard.** Compare the numbers and names in the summary with the article and flag the ones that don’t appear in it.

These are directions, not requirements. One approach done well scores higher than several that half-work.

Resources. XL-Sum, Spanish subset, at huggingface.co/datasets/csebuetnlp/xlsum: BBC Mundo articles, each one with a summary written by a journalist. Use ROUGE (from the `evaluate` library) to compare your summaries with the journalist’s. For the time limit, count about 2 words per second, so 120 words is the maximum.

What to build. An interface where the user pastes an article and gets the summary, its word count and its reading time. Numbers or names in the summary that don’t appear in the article show in red.

Judge test. The judge pastes a story from that day’s local newspaper and reads the summary aloud with a timer.

Why it matters. If a small station had a reliable summarizer, its producers would spend their time checking facts and finding stories. Local news would reach the morning audience faster and with fewer errors.

Judging criteria.

- **Results:** ROUGE on the test set and every summary within 120 words.
- **Faithfulness:** no invented facts in the judge’s stories.
- **Read aloud:** the judge reads the summary in 60 seconds or less and it makes sense.
- **Clarity:** the interface shows clearly what was flagged and why.

Track 3 — From Voice Note to Order

About the company. Ferretería Los Nogales is a hardware store in the Monterrey metropolitan area that delivers to construction sites. (Fictional company.)

The problem today. For years, clients have sent their orders as WhatsApp voice notes. The staff have typed every order by hand, and they have made mistakes with quantities and sizes. Last week a client asked for cable number 12, and the store sent 50 meters of number 10. If someone had confirmed the size, the store wouldn’t have lost that sale.

The big challenge. Can your system turn a voice note in Spanish into an order ticket with the customer’s name, product, quantity, size and delivery time?

Possible approaches.

1. **Transcription.** Use Whisper (`openai/whisper-small` or `openai/whisper-base`), a pre-trained encoder-decoder Transformer with attention. Compare two sizes for accuracy and speed.
2. **Rule baseline.** In this track the baseline is a rule extractor with regular expressions over the transcript, for numbers, units and product words.
3. **Transformer extraction.** A language model that returns the ticket as JSON, or a token-classification model that tags each field.
4. **Catalog matching.** Clients say things like “cable del doce”. Match those words to the closest product in the catalog with sentence embeddings.

These are directions, not requirements. One approach done well scores higher than several that half-work.

Resources. Whisper models on Hugging Face (huggingface.co/openai/whisper-small). Your team records at least 30 voice notes with different voices, speeds and background noise, and writes the correct ticket for each one as the answer key. Invent a catalog of 30 to 50 products in a CSV file. Measure transcription with WER (word error rate, from the `jiwer` or `evaluate` library) and the ticket with the percentage of correct fields.

What to build. An interface where the user records with the microphone or uploads an audio file. It shows the transcript and an editable ticket, and a confirm button adds the order to the day’s list.

Judge test. The judge records a new voice note live in the classroom, with background noise.

Why it matters. If the store read its voice notes automatically, the staff would prepare the order while the client is still on the way. Small businesses would get the same order control that large stores already have.

Judging criteria.

- **Results:** correct fields on the judge’s voice notes.
- **Robustness:** noise, fast speech and different voices.
- **Speed:** seconds from the end of the audio to the ticket on screen.
- **Clarity:** the user can fix a wrong field before confirming.

Track 4 — The Assistant That Doesn’t Make Things Up

About the company. Orienta MX is a citizen-advice service that answers questions about public rules, such as traffic fines and school regulations. (Fictional company.)

The problem today. Many people have asked generic chatbots about fines and school rules. Those chatbots have answered with confidence and have quoted articles that don’t exist. A user recently paid a fine after a chatbot told her it was mandatory. If she had seen the real article, she wouldn’t have paid it.

The big challenge. Can your assistant answer questions about one public regulation, quote the exact article it used, and say clearly when the answer isn’t in the document?

Possible approaches.

1. **TF-IDF baseline.** Split the regulation by article, represent each one with TF-IDF and return the article with the highest cosine similarity.
2. **Semantic retrieval.** Use multilingual sentence embeddings (for example `paraphrase-multilingual-MiniLM-L12-v2`) with a vector index such as FAISS or Chroma.
3. **Answer with sources (RAG).** A language model writes the answer using only the retrieved articles. Use a free API or a local model with Ollama.
4. **Refusal rule.** If the best match is below a similarity threshold, the assistant answers that it can’t find that in the regulation.

These are directions, not requirements. One approach done well scores higher than several that half-work.

Resources. One public regulation in PDF chosen by your team and downloaded from its official website, such as your municipality’s traffic regulation or a UANL school regulation. Build an answer key with 20 questions whose article you know and 10 trap questions whose answer isn’t in the document. Measure retrieval with the percentage of questions where the right article appears in the top 3, and judgment with the percentage of traps the assistant refuses.

What to build. A chat interface in Spanish. Next to every answer, it shows the text of the article it quotes.

Judge test. The judge asks new questions, including trap questions your team hasn’t seen.

Why it matters. If the assistant always showed the article, people would check the answer themselves. A system that refuses to guess earns more trust than one that always has an answer.

Judging criteria.

- **Results:** right article on the judge’s questions.
- **Judgment:** it refuses the trap questions.
- **Faithfulness:** the answer says what the quoted article says.
- **Clarity:** the user can see where every answer comes from.

Track 5 — The Interface That Follows Your Intent

About the company. Casa Norte builds a home assistant in Spanish for families in northern Mexico. (Fictional company.)

The problem today. The assistant has always answered with text. Users have asked it to set alarms and add items to the shopping list, and it has replied with long sentences that they have to read and act on in another app. In user tests, if the assistant had shown a simple card with a button, people would have finished the task in half the time.

The big challenge. Can your system understand what the user wants from one sentence, pick out the key data (time, place, person, item) and show the screen component that completes the task?

Possible approaches.

1. **Intent baseline.** TF-IDF with a classic classifier, or a small LSTM, to choose the intent.
2. **Fine-tuned Transformer.** Fine-tune a Spanish BERT or a multilingual DistilBERT on the intents you choose.
3. **Key data.** Token classification with the slot labels of the dataset, or a language model that returns JSON with the fields.
4. **Components that react.** In Gradio, each intent shows its own card (alarm, weather, shopping list, calendar) and hides the rest. When the user changes a value on the card, the app state changes too.

These are directions, not requirements. One complete flow scores higher than five half-finished screens.

Resources. MASSIVE, locale `es-ES`, at huggingface.co/datasets/AmazonScience/massive, with 60 intents and labeled slots. Choose 6 to 8 intents and train only with those. Measure intent accuracy and F1 on the test set, and the percentage of key data extracted correctly. Add 30 Mexican-style sentences written by your team as an extra test.

What to build. An interface where the user types a sentence. The matching card appears with its data already filled, and at least one action on the card produces a real change, such as an alarm added to the alarm list.

Judge test. The judge types informal sentences your team hasn’t seen, such as “despiértame a las seis y cuarto”.

Why it matters. If assistants built the right screen for each request, people with little experience with apps would complete tasks on their own.

Judging criteria.

- **Results:** right intent and right data on the judge’s sentences.
- **Interface:** the card changes with the intent and its action works.
- **Robustness:** informal Mexican Spanish and sentences outside the chosen intents.
- **Clarity:** the user understands what the assistant understood.